



CS-250: Lab 3

Implementation of Linked List with its Operations

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Class: BESE 16B

Batch: 2k25

Github Repo Link: <https://github.com/mmujtaba25/CS-250>

Zip File for only Lab 03 (Auto generated): <https://github.com/mmujtaba25/CS-250/releases/tag/cs-250-lab-03-540040>

Task 1:

Since the files are large, GitHub links for them are provided below:

See [linked_list.hpp](#) and [linked_list.cpp](#).

Task 2:

See [linked_list.cpp#L76](#)

Task 3:

A ForEach Function was exposed that can be used to print in any way required.

[linked_list.cpp#L226](#)

Usage to print:

[menu.cpp#L60](#)

Task 4:

Using a removelf function we can remove any specific element from the list given a predicate.

[linked_list.cpp#L125](#) (removelf)

See [menu.cpp#L33](#), here we allow the user to give input in the form of “<value>”, “:<index>” or “.” (removes the last element):

[menu.cpp#L184](#) (parsing)

[menu.cpp#L87](#) (removing)

[menu.cpp#L249](#) (user input)

Task 5:

See [linked_list.cpp#L190](#)

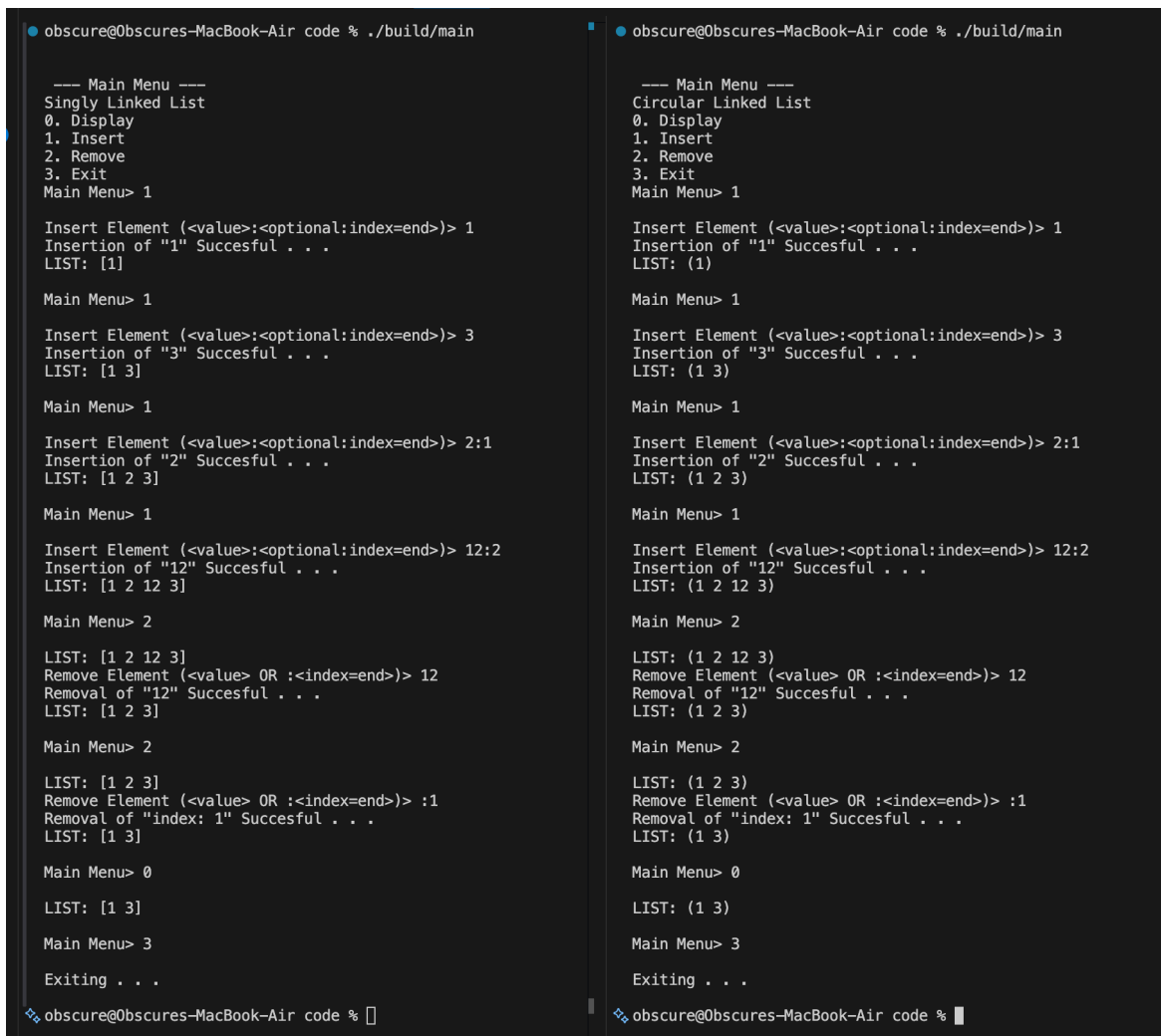
Task 6:

See [menu.cpp](#) and [menu.hpp](#).

Task 7:

We use private inheritance form C++ to make sure only we can call methods for insertion and other stuff from LinkedList class and caller has access to only said function.

See [queue.cpp](#) and [queue.hpp](#).



```
obscura@Obscures-MacBook-Air code % ./build/main
--- Main Menu ---
Singly Linked List
0. Display
1. Insert
2. Remove
3. Exit
Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 1
Insertion of "1" Succesful . . .
LIST: [1]

Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 3
Insertion of "3" Succesful . . .
LIST: [1 3]

Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 2:1
Insertion of "2" Succesful . . .
LIST: [1 2 3]

Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 12:2
Insertion of "12" Succesful . . .
LIST: [1 2 12 3]

Main Menu> 2

LIST: [1 2 12 3]
Remove Element (<value> OR :<index=end>)> 12
Removal of "12" Succesful . . .
LIST: [1 2 3]

Main Menu> 2

LIST: [1 2 3]
Remove Element (<value> OR :<index=end>)> :1
Removal of "index: 1" Succesful . . .
LIST: [1 3]

Main Menu> 0

LIST: [1 3]

Main Menu> 3

Exiting . . .
obscura@Obscures-MacBook-Air code %

obscura@Obscures-MacBook-Air code % ./build/main
--- Main Menu ---
Circular Linked List
0. Display
1. Insert
2. Remove
3. Exit
Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 1
Insertion of "1" Succesful . . .
LIST: (1)

Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 3
Insertion of "3" Succesful . . .
LIST: (1 3)

Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 2:1
Insertion of "2" Succesful . . .
LIST: (1 2 3)

Main Menu> 1

Insert Element (<value>:<optional:index=end>)> 12:2
Insertion of "12" Succesful . . .
LIST: (1 2 12 3)

Main Menu> 2

LIST: (1 2 12 3)
Remove Element (<value> OR :<index=end>)> 12
Removal of "12" Succesful . . .
LIST: (1 2 3)

Main Menu> 2

LIST: (1 2 3)
Remove Element (<value> OR :<index=end>)> :1
Removal of "index: 1" Succesful . . .
LIST: (1 3)

Main Menu> 0

LIST: (1 3)

Main Menu> 3

Exiting . . .
obscura@Obscures-MacBook-Air code %
```

Working Menu, [Left: Singly Linked List] & [Right: Circular Linked List]

Task 8:

For circular linked list, most of the source is the same, except for change requiring the last element to point towards the first element.

See [circular_linked_list.cpp](#) and [circular_linked_list.hpp](#).

Using VS Code Compare we can see the code difference between [[circular_linked_list.cpp](#) and [circular_linked_list.hpp](#)] and [[linked_list.hpp](#) and [linked_list.cpp](#)]

Diff in insertAt Function:

```
72 // MARK: Insert
73 // MARK: Insert
74
75 /// the value is inserted at index 'index', get using 'get(index)';
75 /// the value is inserted at index 'index', get using 'get(index)';
76- bool LinkedList::insertAt(int value, size_t index)
76+ bool CircularLinkedList::insertAt(int value, size_t index)
77 {
77 {
78     if (!indexInBounds(index, true))
78     if (!indexInBounds(index, true))
79         return false;
79         return false;
80
80
81     // inserting into empty list
81     // inserting into empty list
82     // for each cannot run in a empty list
82     // for each cannot run in a empty list
83     if (head == nullptr)
83     if (head == nullptr)
84     {
84     {
85         head = new Node(value, nullptr);
85         head = new Node(value, nullptr);
86         return true;
86+        head->next = head; // point after creation
87         return true;
87
88
89         bool inserted = false;
89         bool inserted = false;
90
90
91     // inserting at the end
91     // inserting at the start of end
92     if (index == count())
92+ if (index == 0 || index == count())
93     {
93+ {
94         forEach([&value, &inserted](NodeLink current, size_t index) { //
94+ forEach([this, &value, &index, &inserted](NodeLink current, size_t index) { //
95             if (current->next == nullptr && !inserted)
95+ if (current->next == head && !inserted)
96             {
96+ {
97                 current->next = new Node(value, nullptr);
97+ current->next = new Node(value, head);
98
98+
99
99+
100
100+
101+ // new node becomes the first one
101+ if (index == 0)
102+     head = current->next;
102
102
103         // make sure executes once
103         // make sure executes once
104         inserted = true;
104         inserted = true;
105
105
106     });
106
107
107+
108
108         return inserted;
108         return inserted;
109
109
110
110
111
111
112
112
113
113
114
114
115
115
116
116
117
117
118
118
119
119
120
120
121
121
122
122
123
123
124
124
125
125
126
126
127
127
```

Diff in removelf Function:

```
122 // MARK: Remove
123
124- bool LinkedList::removeIf(const std::function<bool(const int &, size_t)> &predicate)
125 {
126     // nothing to remove
127     if (head == nullptr)
128         return false;
129
130     // check if predicate applies to head
131     if (predicate(head->data, 0))
132     {
133         Node *old = head;
134         // move head by one
135         head = head->next;
136
137         // free up old space
138         delete old;
139
140         return true;
141     }
142
143     bool removed = false;
144     // check predicate : all except head
145-    forEach([&predicate, &removed](NodeLink current, size_t index) { //
146        // run only once && not empty or last
147-        if (removed || current == nullptr || current->next == nullptr)
148            return; // equivalent to continue in a for loop
149
150        // check if predicate applies to the next node
151        Node *nextNode = current->next;
152        if (predicate(nextNode->data, index + 1))
153        {
154            // point current's next to the next, next node
155            current->next = current->next->next;
156            // delete nextNode
157            delete nextNode;
158
159            removed = true;
160        }
161    });
162
163    return removed;
164 }
165
```

```
127 // MARK: Remove
128
129- bool CircularLinkedList::removeIf(const std::function<bool(const int &, size_t)> &predicate)
130 {
131     // nothing to remove
132     if (head == nullptr)
133         return false;
134
135     // check if predicate applies to head
136     if (predicate(head->data, 0))
137     {
138         Node *old = head;
139         // move head by one; given head doesn't point it itself
140         // if it does, then one element list, and removing the list
141         head = (head->next == head ? nullptr : head->next);
142
143         // update reference to head for the last node
144         forEach([this, &old](NodeLink current, size_t) { //
145             if (current->next == old)
146                 current->next = head;
147         });
148
149         // free up old space
150         delete old;
151
152         return true;
153     }
154
155     bool removed = false;
156     // check predicate : all except head
157-    forEach([this, &predicate, &removed](NodeLink current, size_t index) { //
158        // run only once && not empty or last
159-        if (removed || current == nullptr || current->next == head)
160            return; // equivalent to continue in a for loop
161
162        // check if predicate applies to the next node
163        Node *nextNode = current->next;
164        if (predicate(nextNode->data, index + 1))
165        {
166            // point current's next to the next, next node
167            current->next = current->next->next;
168            // delete nextNode
169            delete nextNode;
170
171            removed = true;
172        }
173    });
174
175    return removed;
176 }
177
```

Diff in removeAll Function:

```
176- void LinkedList::removeAll()
177 {
178     // keep deleting until head is null as well
179     // i.e memory at head is free as well
180     while (head != nullptr)
181     {
182         Node *next = head->next;
183         delete head;
184         head = next;
185     }
186 }
187
```

```
189+ void CircularLinkedList::removeAll()
190 {
191+     // already empty
192+     if (head == nullptr)
193+         return;
194+
195+     // unwrap the circle: head becomes the last node, pointing to null
196+     Node *second = head->next;
197+     head->next = nullptr;
198+     head = second;
199+
200     // keep deleting until head is null as well
201     // i.e memory at head is free as well
202     while (head != nullptr)
203     {
204         Node *next = head->next;
205         delete head;
206         head = next;
207     }
208 }
209
```

Diff in forEach Functions:

```
200 // MARK: For Each
201 // MARK: For Each
202 // MARK: For Each
203 /// @brief Runs between index 0 to 'count()' - 1
204 ///
205 /// MUTABLE
206 /// @param func '(NodeLink current, size_t currentIndex) -> void'
207 void LinkedList::forEach(std::function<void(NodeLink, size_t)> func)
208 {
209     size_t index = 0;
210     Node *current = &head;
211     while (*current != nullptr)
212     {
213         // pass reference to the node itself
214         func(*current, index);
215
216         // move to next value
217         current = &(*current->next);
218         index++;
219     }
220 }
221
222 /// @brief Runs between index 0 to 'count()' - 1
223 ///
224 /// IMMUTABLE
225 /// @param func '(ConstNode current, size_t currentIndex) -> void'
226 void LinkedList::forEach(std::function<void(ConstNode, size_t)> func) const
227 {
228     size_t index = 0;
229     Node *current = head;
230     while (current != nullptr)
231     {
232         // pass const "copy" of the node
233         func(current, index);
234
235         // move to next value
236         current = current->next;
237         index++;
238     }
239 }
240
222 // MARK: For Each
223 // MARK: For Each
224 // MARK: For Each
225 /// @brief Runs between index 0 to 'count()' - 1
226 ///
227 /// MUTABLE
228 /// @param func '(NodeLink current, size_t currentIndex) -> void'
229 void CircularLinkedList::forEach(std::function<void(NodeLink, size_t)> func)
230 {
231     if (head == nullptr)
232         return;
233
234     size_t index = 0;
235     Node *current = &head;
236     do // do while because we start at head
237     {
238         // pass reference to the node itself
239         func(*current, index);
240
241         // move to next value
242         current = &(*current->next);
243         index++;
244     } // continue until head shows up again
245     while (*current != head);
246 }
247
248 /// @brief Runs between index 0 to 'count()' - 1
249 ///
250 /// IMMUTABLE
251 /// @param func '(ConstNode current, size_t currentIndex) -> void'
252 void CircularLinkedList::forEach(std::function<void(ConstNode, size_t)> func) const
253 {
254     if (head == nullptr)
255         return;
256
257     size_t index = 0;
258     Node *current = head;
259     do // do while because we start at head
260     {
261         // pass const "copy" of the node
262         func(current, index);
263
264         // move to next value
265         current = current->next;
266         index++;
267     } // continue until head shows up again
268     while (current != head);
269 }
270
```